Presentation 1

Link to presentation PDF, and code:

- Mathletes write-up https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1/Mathletes_Writeup_Presentation1_Junho.md
- Code1 (fraction of patches below 32x32 px) https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1/Code1_PatchSizeFilter_22Oct_Junho.py
- Code2 (parameter counts and class-weight check) https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1/Code2_MathChecks_22Oct_Junho.py
- Link to presentation:

  https://docs.google.com/presentation/d/1ZgIi25akCi9E1mNgbOpdod1Ui3sWDL4o9EvB1rbrWFE/edit?usp=sharing

- Link to github repo:

  https://github.com/arvindkrishna87/Team1_FullStack/tree/main/Presentation1_base_code

    - All code references here are to code already implemented in the model, and no code has been added.

What did each of the team members do?

Grace and Sarah worked as consultants, capturing the current methodology and exploring potential recommendations using available literature. Grace and Sarah worked through the four presented stages, including patching the slices, embedding the patches, splitting by case, and then training and evaluating. They split the workforce into six steps and created flowcharts for each. In addition, they made justifications for why certain decisions were made and what they contribute to the model and predictions. Furthermore, Grace and Sarah read up on current literature and made a proposal to try a new model, UNI or UNI 2, to embed the patches.

Junho worked as the Mathlete and wrote the whole pipeline down as mathematics, from data preprocessing to the loss. This covers the label map, the case, stain, slice and patch hierarchy with its caps, the 32x32 patch filter, the image normalisation, the case-level fold constraint, the tensor shapes through the hierarchical attention model, the parameter count, the loss and every regulariser. From these equations he found four issues, each with a suggested fix. Hyperparameters were tuned on folds that later serve

as test folds, the class weight cancels at batch size 1, the entropy bonus and weight decay are too small to have an effect, and the test set is scored with last-epoch weights. He also showed that a test fold of 18-19 cases is too small to confirm a 95% target.

Ashley and Nolan worked as the Analysts, investigating what the model looks at when it makes a prediction. They walked through attention_analysis.py and documented how it combines stain, slice and patch attention into an effective score and maps the highest- and lowest-scoring patches back to their tissue images, summarizing the logic in an overview and a flowchart. They then ran the analysis on split 1 and visualized high- and low-attention patches, with their attention values, for the most confident case in each classification category, along with the distribution of patch attention per case. Split 1 had no misclassified high-grade cases. Finally, they critically reflected on the method and identified two flaws, each with suggested improvements.

Issues faced that haven't been resolved:

Junho: How much Issue 1 inflates the reported accuracy cannot be computed from the code and would need a nested cross-validation run.

Sarah and Grace did not face any issues that have not been addressed.

As per the task at hand, Ashley and Nolan noted a few flaws in the attention analysis process, but had no other issues.

Proposed next steps: (not required)

As the consultants, Sarah and Grace recommend trying to use models UNI2/UNI to extract the feature vectors from the patches instead of KimiaNet.

References

Grace and Sarah built on the methodology that has been created by STAT 390 students over the past two years. This includes the data preprocessing and architecture exploration done in the 2024-25 year and model development, training, and testing done in the 2025-26 year.

Junho built on the base code and Code walkthrough slides from the STAT390 instruction team, which come from the work of the 2024-25 and 2025-26 cohorts. Literature used includes Ilse et al. (2018) on attention-based MIL, Varma and Simon

(2006) on tuning bias, Zhang et al. (AEM, 2024) on attention entropy, Loshchilov and Hutter (2019) on AdamW, and Varoquaux (2018) on cross-validation error bars. Claude (Anthropic) helped read the code.

Ashley and Nolan built on the base code (attention_analysis.py) and the code walkthrough materials in lecture, which draw on the model development, training and testing work of the STAT 390 2024-25 and 2025-26 cohorts. The analysis used the trained model and test outputs from the split 1 run located at /projects/e32998/MIL_training/final_runs/run_20261022_115349/.